arXiv:2306.00029v2 [cs.SE] 22 Dec 2025

CODETF: ONE-STOP TRANSFORMER LIBRARY FOR STATE-OF-THE-ART CODE LLMS

Nghi D. Q. Bui*, **Hung Le, Yue Wang, Junnan Li, Akhilesh Deepak Gotmare, Steven C.H. Hoi***
Salesforce AI Research
https://github.com/salesforce/CodeTF

ABSTRACT

Large language models (LLMs) have revolutionized code intelligence, yet practitioners face persistent challenges: fragmented interfaces across models, code-specific preprocessing requirements for diverse programming languages, and a lack of standardized evaluation protocols that hinders reproducibility.

We present CodeTF, an open-source Transformer library for Code LLMs that provides a unified interface supporting 9+ pretrained models across three architectures, code utilities for 15+ programming languages with built-in AST parsing, and standardized evaluation on benchmarks including HumanEval, MBPP, and APPS. The library offers quantization support, parameter-efficient fine-tuning, and comprehensive data preprocessing pipelines.

We describe the design principles, modular architecture, and key components of CodeTF, demonstrating how it simplifies the complete workflow from data preparation to model deployment. CodeTF bridges the gap between machine learning research and software engineering practice, enabling both researchers and practitioners to efficiently develop, evaluate, and deploy Code LLMs.

Keywords Transformer · code large language models · code understanding · code generation · code intelligence

1 Introduction

The integration of artificial intelligence into software engineering has accelerated dramatically in recent years, fundamentally transforming how developers write, review, and maintain code. Traditional approaches to code intelligence relied on rule-based static analysis and basic machine learning techniques [1, 2, 3]. Today, Transformer-based large language models (LLMs) pretrained on massive code corpora—commonly referred to as "Code LLMs"—have achieved unprecedented performance on tasks ranging from code generation and completion to program repair and code summarization [4, 5, 6, 7, 8, 9, 10].

The commercial impact of these advances has been substantial. GitHub Copilot, powered by OpenAI's Codex, has been adopted by millions of developers worldwide, demonstrating that Code LLMs can significantly boost developer productivity. Research models such as CodeGen [5], StarCoder [11], and CodeT5+ [12] have pushed the boundaries of what is possible, achieving impressive results on benchmarks like HumanEval [13] and APPS [14]. These models learn rich representations of code by training on billions of tokens from open-source repositories on platforms like GitHub [5, 6, 10], often incorporating programming language features such as data flow and control flow [4, 9, 7].

Despite these advances, practitioners and researchers face significant challenges when working with Code LLMs:

1. **Fragmented Interfaces:** Each model family (CodeBERT, CodeT5, CodeGen, StarCoder) comes with its own API, configuration format, and usage patterns. Switching between models or conducting fair comparisons requires substantial engineering effort to adapt data pipelines and training scripts.
2. **Code-Specific Preprocessing:** Unlike natural language, source code has strict syntactic rules that vary across programming languages. Effective preprocessing often requires Abstract Syntax Tree (AST) parsing, identifier extraction, and language-specific tokenization—capabilities not provided by general-purpose NLP libraries.

*Correspondence: {nghi.bui, shoi}@salesforce.com

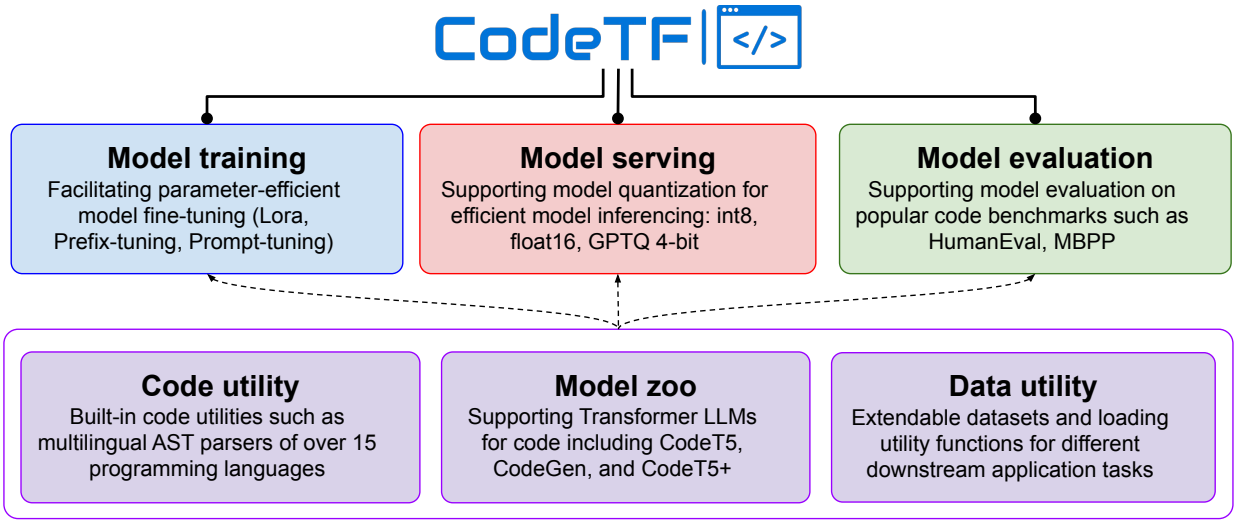

Figure 1: **An overview of CodeTF:** A comprehensive Transformer-based library supporting the complete workflow for Code LLMs—from data preparation and model training to serving and evaluation. The library integrates pretrained models, code utilities for 15+ programming languages, and standardized benchmarks into a unified framework.

3. **Reproducibility Crisis:** Many published Code LLM results are difficult to reproduce due to inconsistent evaluation protocols, missing preprocessing scripts, and variations in benchmark implementations. This hinders progress and makes it challenging for researchers to build upon prior work.

To address these challenges, we introduce **CodeTF**, an open-source, comprehensive library that provides a unified framework for developing, training, serving, and evaluating Code LLMs. Figure 1 presents an overview of the library architecture. The key insight behind CodeTF is that code intelligence tasks share common infrastructure needs—model loading, quantization, fine-tuning, code parsing, and evaluation—that can be abstracted into reusable, well-designed modules.

CodeTF supports a diverse collection of pretrained models spanning three major architectures: encoder-only models such as CodeBERT [6] and CodeBERTa [15], which are optimized for code understanding tasks; decoder-only models including CodeParrot [16], InCoder [17], CodeGen [5], SantaCoder [18], and StarCoder [11], designed for code generation; and encoder-decoder models like CodeT5 [4] and CodeT5+ [12], which offer versatility for both understanding and generation.

The library also provides built-in support for popular benchmarks including HumanEval, APPS, MBPP, The Vault [13, 14, 19, 20, 21], and CodeXGLUE, with standardized data loaders and evaluation metrics. Through a unified interface, users can load any supported model with a single function call, fine-tune it using parameter-efficient methods (LoRA, Prefix-Tuning, P-Tuning), and evaluate it on multiple benchmarks—all with consistent APIs.

A distinguishing feature of CodeTF is its comprehensive code utility module. Unlike general NLP libraries, CodeTF provides built-in AST parsing for 15+ programming languages using tree-sitter, enabling extraction of code attributes such as function names, identifiers, comments, and control flow structures. These capabilities are essential for tasks like CodeT5's identifier-aware pretraining [4] and are often the most time-consuming aspect of code intelligence research to implement from scratch.

In summary, CodeTF makes four main contributions. First, it provides a **unified model interface**—a consistent API for loading, serving, and fine-tuning 9+ pretrained Code LLMs across three architectures, with built-in quantization support (8-bit, 4-bit) for efficient deployment. Second, it offers **comprehensive code utilities** including AST parsing and code attribute extraction for 15+ programming languages, eliminating the need for researchers to build custom preprocessing pipelines. Third, it enables **standardized evaluation** with built-in support for popular benchmarks (HumanEval, MBPP, APPS, CodeXGLUE) and consistent metrics (pass@k, CodeBLEU, Edit Similarity), facilitating reproducible comparisons. Fourth, it integrates **parameter-efficient fine-tuning** through HuggingFace PEFT, enabling memory-efficient adaptation of large models using LoRA, Prefix-Tuning, and other techniques. The library is accompanied by extensive documentation, tutorials, and example scripts to help users get started quickly.

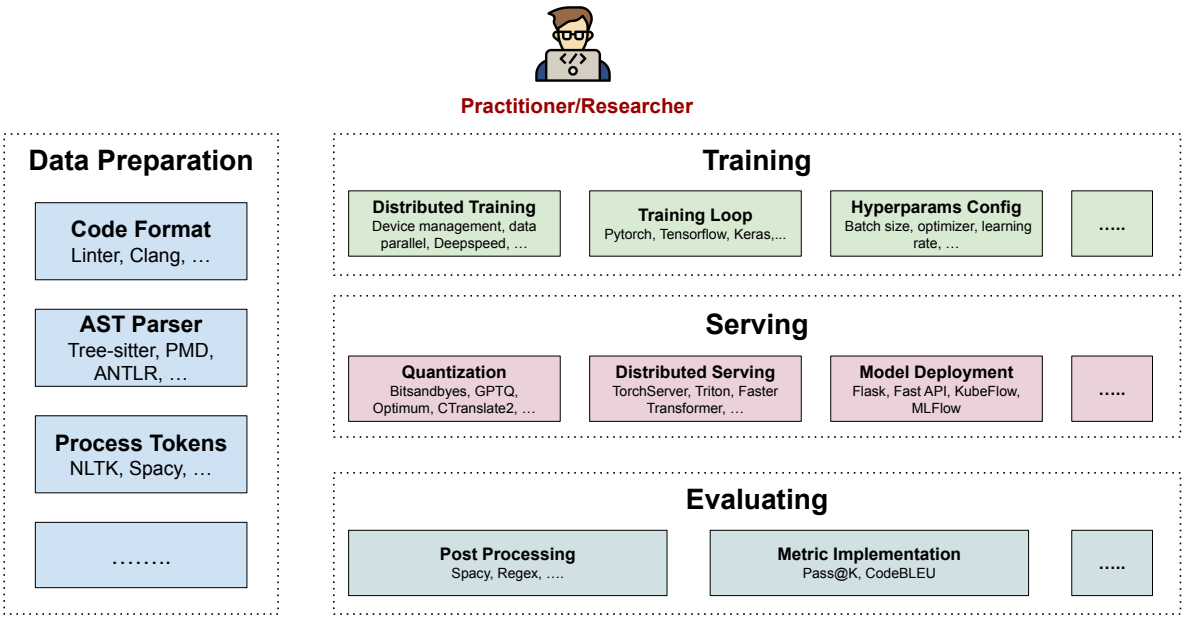

Figure 2: Common workflows when working with Code LLMs. Practitioners typically need to handle four interconnected tasks: data preparation, model training/fine-tuning, model serving, and evaluation. CodeTF provides unified support for all four stages.

CodeTF is designed to serve both researchers exploring new Code LLM architectures and practitioners deploying models in production. By providing a comprehensive, unified, and extensible framework, we aim to accelerate progress in code intelligence research and broaden the adoption of Code LLMs in software development workflows. The following section describes the library's design principles and modular architecture.

2 Library Design

Figure 3 provides a detailed overview of the CodeTF system architecture, illustrating how the modular design enables users to seamlessly combine different components for their specific code intelligence workflows.

2.1 Motivation

To understand the design rationale behind CodeTF, consider the typical workflow of practitioners working with Code LLMs (Figure 2). This workflow involves four interconnected tasks, each presenting unique challenges:

Data Preparation. Before training or fine-tuning Code LLMs, practitioners must prepare their data through multiple preprocessing steps. This includes formatting source code (using tools like clang-format or ESLint), parsing code into Abstract Syntax Trees (using tree-sitter or language-specific parsers), and extracting relevant features such as function names, identifiers, and comments. These preprocessing steps are crucial—models like CodeT5 require identifier locations for their pretraining objectives, and defect prediction tasks benefit from AST-based features. However, implementing these pipelines from scratch is time-consuming and error-prone.

Training and Fine-Tuning. Once data is prepared, users must configure training pipelines including optimizer settings, learning rate schedules, and distributed training across multiple GPUs. For large models (billions of parameters), full fine-tuning may be infeasible, requiring parameter-efficient methods like LoRA or Prefix-Tuning. Each model family often requires different training configurations, making it difficult to experiment across architectures.

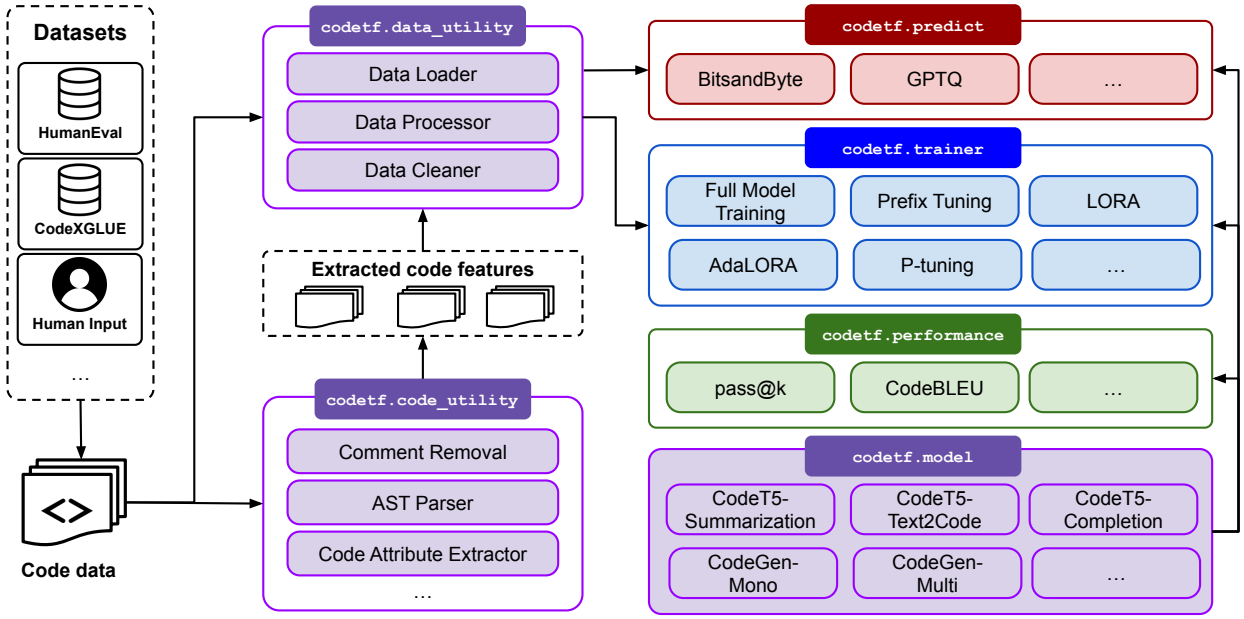

Figure 3: System architecture of CodeTF showing the six core modules: Model Zoo (`codetf.model`), Model Serving (`codetf.predict`), Model Training (`codetf.trainer`), Data Utility (`codetf.data_utility`), Code Utility (`codetf.code_utility`), and Evaluator (`codetf.performance`).

Model Serving. Deploying trained models for inference presents additional challenges. Large models require quantization (8-bit or 4-bit) to fit in memory and achieve acceptable latency. Users must also handle batching, input preprocessing, and output post-processing consistently across different model architectures.

Evaluation. Finally, evaluating models on standard benchmarks requires implementing task-specific metrics (pass@k for code generation, CodeBLEU for translation) and handling benchmark-specific data formats. Inconsistent implementations across papers have led to reproducibility issues in the field.

Existing libraries like HuggingFace Transformers provide excellent general-purpose infrastructure but lack code-specific features such as AST parsing, code-aware tokenization, and specialized evaluation metrics. CodeTF addresses this gap by providing a higher-level abstraction layer specifically designed for code intelligence tasks.

2.2 Design Principles

CodeTF is built around six core design principles that guide its architecture and API design:

1. **Comprehensiveness.** CodeTF aims to be a one-stop solution covering the complete Code LLM workflow. The library supports model loading and serving across three architectures (encoder-only, decoder-only, encoder-decoder), multiple fine-tuning strategies (full fine-tuning, LoRA, Prefix-Tuning), comprehensive evaluation metrics, and code utilities for 15+ programming languages. Users should rarely need to integrate external tools for standard code intelligence tasks.
2. **Unified Interface.** Despite the diversity of supported models and tasks, CodeTF provides consistent APIs across all components. Loading any model requires a single function call (`load_model_pipeline`) with the same signature regardless of architecture. Training, evaluation, and code parsing follow similar patterns, reducing the learning curve and enabling rapid experimentation across different models.
3. **Ease of Use.** CodeTF prioritizes developer experience with minimal setup requirements. Installation via pip or conda provides all dependencies, and users can start with pretrained models in just a few lines of code. Sensible defaults are provided for all configurations, while advanced users can customize any aspect of the pipeline.
4. **Extensibility.** The code intelligence field evolves rapidly, with new models and benchmarks emerging frequently. CodeTF follows object-oriented design principles, with abstract base classes (`BaseTrainer`, `BaseCodeUtility`) that can be extended for new model architectures or programming languages. Adding support for a new model requires implementing a small number of well-defined interfaces.

5. **Scalability.** Training and serving large Code LLMs requires efficient resource utilization. CodeTF integrates with distributed training frameworks (DeepSpeed, FSDP) and provides built-in quantization support (BitsAndBytes, GPTQ) to enable deployment on consumer hardware. The library handles multi-GPU training configuration automatically.
6. **Reproducibility.** To address the reproducibility crisis in Code LLM research, CodeTF provides standardized implementations of popular benchmarks and metrics. All evaluation code is versioned and tested, ensuring that results obtained with CodeTF can be reliably reproduced. We also provide reference results for supported models on standard benchmarks.

2.3 Module Overview

Based on these principles, CodeTF is organized into six interconnected modules. The **Model Zoo** (`codetf.model`) serves as a repository of pretrained and fine-tuned model configurations, supporting encoder-only models (CodeBERT, CodeBERTa), decoder-only models (CodeGen, StarCoder, SantaCoder, InCoder, CodeParrot), and encoder-decoder models (CodeT5, CodeT5+). The **Model Serving** module (`codetf.predict`) provides a unified inference interface with built-in quantization (8-bit, 4-bit), batching, and preprocessing for tasks including code generation, completion, and summarization. The **Model Training** module (`codetf.trainer`) offers training utilities supporting both full fine-tuning and parameter-efficient methods (LoRA, Prefix-Tuning, P-Tuning, AdaLoRA) with automatic GPU management and checkpoint saving. For data handling and evaluation, the **Data Utility** module (`codetf.data_utility`) provides data loaders for popular benchmarks (HumanEval, APPS, MBPP, CodeXGLUE, The Vault) along with preprocessing pipelines for tokenization and formatting. The **Code Utility** module (`codetf.code_utility`) enables AST parsing for 15+ programming languages using tree-sitter, with utilities for extracting function names, identifiers, comments, and other code attributes. Finally, the **Evaluator** module (`codetf.performance`) implements standardized evaluation metrics including pass@k, CodeBLEU, Edit Similarity, and BLEU, with benchmark-specific evaluation scripts. The following section provides detailed descriptions of each module's capabilities and implementation.

3 Modules and Utilities

This section provides detailed descriptions of each module in CodeTF, explaining their capabilities, design choices, and usage patterns.

3.1 Model Zoo

The Model Zoo (`codetf.model`) serves as the central repository for model configurations in CodeTF. It provides standardized access to pretrained and fine-tuned checkpoints from state-of-the-art Code LLMs, abstracting away the complexity of different model architectures and hosting platforms.

CodeTF supports three major families of Transformer architectures:

- **Encoder-only models** (CodeBERT [6], CodeBERTa [15]): These models excel at code understanding tasks such as code search, clone detection, and defect prediction. They encode bidirectional context, making them suitable for classification and retrieval tasks.
- **Decoder-only models** (CodeParrot [16], InCoder [17], CodeGen [5], SantaCoder [18], StarCoder [11]): Optimized for code generation through autoregressive next-token prediction. These models range from 350M to 16B parameters, offering trade-offs between capability and resource requirements.
- **Encoder-decoder models** (CodeT5 [4], CodeT5+ [12]): Versatile architectures suitable for both understanding and generation tasks, including code summarization, translation, and refinement.

Each model is accompanied by a YAML configuration file specifying the HuggingFace repository URL, tokenizer settings, maximum sequence length, and task-specific parameters. This design enables CodeTF to stay current with the latest model releases simply by adding new configuration files, without requiring code changes.

3.2 Model Serving Module

The Model Serving module (`codetf.predict`) provides a unified interface for loading models and performing inference. As illustrated in Figure 4, the module uses a registry pattern to dispatch model loading requests to the appropriate model class based on the specified architecture.

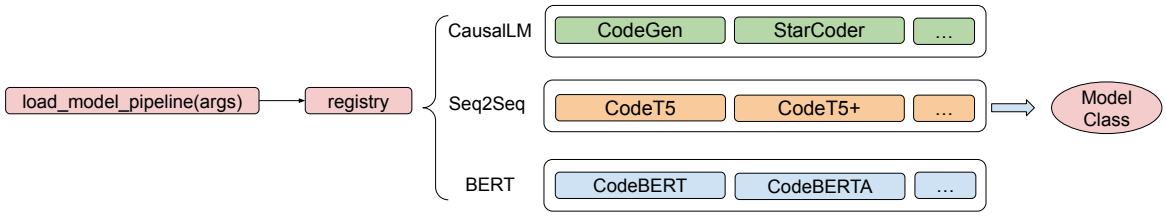

Figure 4: Model loading pipeline in CodeTF. The `load_model_pipeline` function provides a unified entry point that dispatches to architecture-specific model classes (`CausalLMModel`, `Seq2SeqModel`, `BERTModel`) based on the requested model type.

Unified Loading Interface. Users can load any supported model with a single function call:

```
model = load_model_pipeline(
    model_name="codet5-base",
    task="code_summarization",
    language="python"
)
```

The function automatically handles tokenizer initialization, model weight loading, and device placement (CPU/GPU).

Quantization Support. Large models like CodeGen-16B or StarCoder-15B require significant memory and can be slow for inference. CodeTF integrates two quantization backends to address this: BitsAndBytes [22], which enables 8-bit and 4-bit quantization with minimal accuracy loss, reducing memory requirements by 2-4x; and GPTQ [23], which provides post-training quantization for even greater compression, particularly useful for deployment on consumer GPUs. In our benchmarks, 8-bit quantization reduces inference time for CodeGen-16B from approximately 1.2 seconds to 0.4 seconds per sample while maintaining 98% of the original accuracy.

3.3 Model Training Module

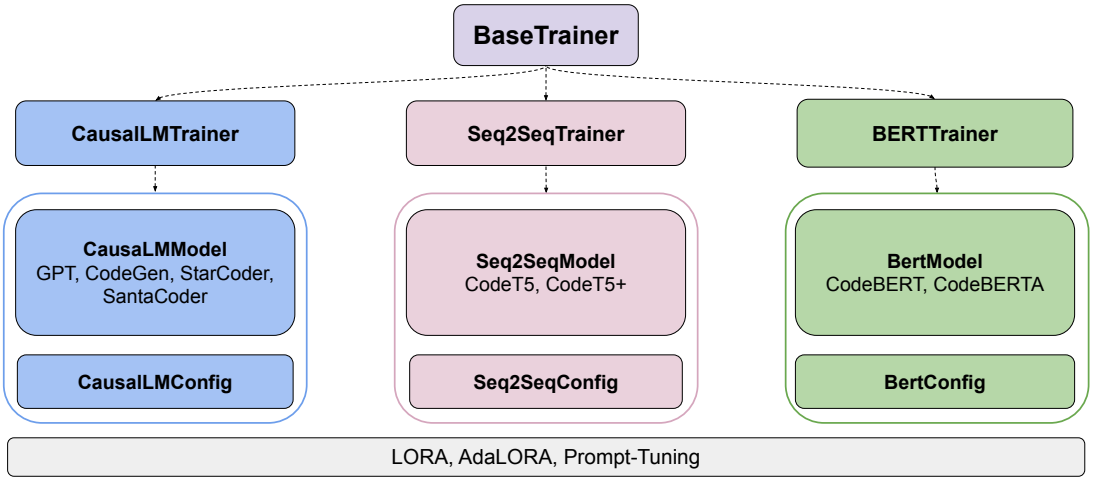

Figure 5: Training module architecture. The `BaseTrainer` class provides common functionality (checkpoint saving, logging, GPU management) inherited by architecture-specific trainers (`CausalLMTrainer`, `Seq2SeqTrainer`, `BERTTrainer`).

The Training Module (`codetf.trainer`) provides utilities for fine-tuning pretrained models on custom datasets. As shown in Figure 5, the module follows an inheritance hierarchy with `BaseTrainer` providing common functionality and specialized trainers handling architecture-specific details.

Full Fine-Tuning. For users with sufficient computational resources, CodeTF supports standard full fine-tuning with configurable learning rates, batch sizes, and optimization strategies. The trainer automatically handles multi-GPU training with data parallelism, gradient accumulation for large effective batch sizes, learning rate scheduling (linear, cosine, polynomial decay), checkpoint saving and resumption, and integration with Weights & Biases for experiment tracking.

Parameter-Efficient Fine-Tuning. For large models or limited GPU memory, CodeTF integrates HuggingFace's PEFT library [2] to enable parameter-efficient fine-tuning methods:

- **LoRA** [24]: Adds low-rank adaptation matrices to attention layers, typically training only 0.1-1% of parameters.
- **Prefix-Tuning** [25]: Prepends trainable prefix tokens to each layer's key and value matrices.
- **P-Tuning** [26]: Uses trainable continuous prompts for task adaptation.
- **AdaLoRA** [27]: Dynamically allocates parameter budget across layers based on importance.

These methods enable fine-tuning models with billions of parameters on a single consumer GPU while achieving competitive performance with full fine-tuning.

3.4 Data Utility Module

The Data Utility module (`codetf.data_utility`) provides comprehensive data loading and preprocessing capabilities for code intelligence tasks.

Benchmark Data Loaders. CodeTF includes built-in data loaders for popular code intelligence benchmarks. These include HumanEval [13], which contains 164 hand-written Python programming problems for evaluating code generation; MBPP [20], with 974 Python programming problems covering basic programming concepts; APPS [14], offering 10,000 coding problems of varying difficulty levels with test cases; CodeXGLUE [19], a benchmark suite covering 10 code intelligence tasks across multiple languages; and The Vault [21], a comprehensive multilingual dataset for code understanding and generation. Each data loader provides consistent interfaces for accessing training, validation, and test splits, with automatic downloading and caching of datasets.

Preprocessing Pipelines. The module offers preprocessing utilities for common code intelligence tasks, including tokenization that wraps HuggingFace tokenizers with code-aware preprocessing (handling special tokens and truncation strategies), code formatting to normalize style and reduce spurious variations in training data, data augmentation with transformations like variable renaming and dead code insertion for robustness training, and batch collation that handles dynamic batching with efficient padding for variable-length code sequences.

3.5 Evaluator Module

The Evaluator Module (`codetf.performance`) provides standardized implementations of evaluation metrics commonly used in code intelligence research.

Execution-Based Metrics. For code generation tasks, CodeTF implements the pass@k metric [13, 28], which measures the probability that at least one of k generated samples passes all test cases. The evaluator handles secure code execution in sandboxed environments, timeout handling for infinite loops, multi-language support (Python, Java, JavaScript, C++), and parallel evaluation for efficiency.

Reference-Based Metrics. For tasks with reference outputs (summarization, translation), CodeTF provides CodeBLEU [29], a variant of BLEU that incorporates code-specific features including AST matching and data flow analysis; Edit Similarity [30], which measures the Levenshtein distance between generated and reference code; and standard NLP metrics such as BLEU and ROUGE for measuring n-gram overlap. All metrics are implemented with consistent interfaces, enabling easy comparison across different models and tasks.

3.6 Code Utility Module

The Code Utility module (`codetf.code_utility`) provides language-aware tools for parsing and manipulating source code—a capability that distinguishes CodeTF from general-purpose NLP libraries.

[2] https://github.com/huggingface/peft

AST Parsing. CodeTF integrates tree-sitter[3], an incremental parsing library, to provide AST parsing for 15+ programming languages: Java, Python, JavaScript, C, C++, C#, Go, Ruby, Rust, PHP, Kotlin, Scala, Haskell, Apex, SOQL, SOSL, Solidity, and YAML. Tree-sitter grammars are precompiled into platform-specific binaries ('.so' files) for Darwin, Linux, and Windows, eliminating the need for users to compile parsers themselves.

Code Attribute Extraction. Building on the AST infrastructure, the module provides utilities for extracting common code attributes, including identifiers (variable names, function names, class names), comments (docstrings, inline comments, block comments), structural elements (function boundaries, class hierarchies, import statements), and control flow constructs (loop structures, conditional branches, exception handling). These utilities are essential for preprocessing pipelines—for example, CodeT5's pretraining requires masking identifier tokens, which requires accurate identifier extraction.

Language-Specific Utilities. Each supported language has a dedicated utility class (e.g., `PythonCodeUtility`, `JavaCodeUtility`) that inherits from `BaseCodeUtility`. This design allows language-specific customizations while maintaining a consistent interface across languages.

The following section demonstrates typical usage patterns for these modules through concrete code examples.

4 Example Usage

This section demonstrates CodeTF's capabilities through practical code examples covering the three most common use cases: model inference, fine-tuning, and evaluation.

Model Loading and Inference. The following example shows how to load a pretrained CodeT5 model for code summarization and generate a summary for a Python function. With just a few lines of code, users can load a model and perform inference:

```python
from codetf.models import load_model_pipeline

summarization_model = load_model_pipeline(model_name="codet5", task="sum-python",
            model_type="base", is_eval=True,
            load_in_8bit=True, weight_sharding=False)

code_snippets = """
    void bubbleSort(int arr[])
    {
        int n = arr.length;
        for (int i = 0; i < n - 1; i++)
            for (int j = 0; j < n - i - 1; j++)
                if (arr[j] > arr[j + 1]) {
                    // swap arr[j+1] and arr[j]
                    int temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                }
    }
"""

# Bubble sort program to sort an integer array
summaries = summarization_model.predict([code_snippets])
```

The `load_model_pipeline` function automatically handles model downloading, tokenizer initialization, and GPU placement. The returned model object provides a simple `predict` method that accepts raw code as input and returns the generated summary.

Fine-Tuning with Parameter-Efficient Methods. CodeTF simplifies the fine-tuning process by providing pre-configured trainers for each model architecture. The following example demonstrates fine-tuning a CodeGen model on the CodeXGLUE code refinement dataset using LoRA for memory efficiency:

[3] https://github.com/tree-sitter/tree-sitter

```python
from codetf.trainer.causal_lm_trainer import CausalLMTrainer
from codetf.data_utility.codexglue_dataset import CodeXGLUEDataset
from codetf.models import load_model_pipeline
from codetf.performance.evaluate import EvaluationMetric

model_class = load_model_pipeline(model_name="causal-lm", task="pretrained",
                model_type="codegen-350M-mono", is_eval=False,
                load_in_8bit=False, weight_sharding=False)


dataloader = CodeXGLUEDataset(tokenizer=model_class.get_tokenizer())
train_dataset, test_dataset, val_dataset = dataloader.load(subset="text-to-code")

evaluator = EvaluationMetric(metric="bleu", tokenizer=model_class.tokenizer)

# peft can be in ["lora", "prefixtuning"]
trainer = CausalLMTrainer(train_dataset=train_dataset,
                        validation_dataset=val_dataset,
                        peft=None,
                        pretrained_model_or_path=model_class.get_model(),
                        tokenizer=model_class.get_tokenizer())
trainer.train()
```

The trainer handles data loading, tokenization, gradient accumulation, and checkpoint saving automatically. Users can customize training hyperparameters through the configuration dictionary or use sensible defaults.

Standardized Evaluation. CodeTF provides unified evaluation interfaces for popular benchmarks. The following example evaluates a CodeGen model on the HumanEval benchmark using the pass@k metric:

```python
from codetf.models import load_model_pipeline
from codetf.data_utility.human_eval_dataset import HumanEvalDataset
from codetf.performance.model_evaluator import ModelEvaluator

os.environ["HF_ALLOW_CODE_EVAL"] = "1"
os.environ["TOKENIZERS_PARALLELISM"] = "true"

model_class = load_model_pipeline(model_name="causal-lm", task="pretrained",
            model_type="codegen-350M-mono", is_eval=True,
            load_in_8bit=True, weight_sharding=False)

dataset = HumanEvalDataset(tokenizer=model_class.get_tokenizer())
prompt_token_ids, prompt_attention_masks, references= dataset.load()

problems = TensorDataset(prompt_token_ids, prompt_attention_masks)

evaluator = ModelEvaluator(model_class)
pass_at_k = evaluator.evaluate_pass_k(problems=problems, unit_tests=references, k
    =[1,10,100])
```

The evaluator handles code generation, test case execution, and metric computation. It supports parallel evaluation and provides detailed per-problem results in addition to aggregate metrics.

5 Related Work

This section provides an overview of research on LLMs for code, discusses commercial code assistants, and compares CodeTF with related libraries and tools.

Large Language Models for Code. The application of large language models to code has grown rapidly, driven by their success on a wide range of tasks including code generation [6, 4, 31, 32], code completion [6, 4, 33], program repair [34], and code translation [35, 36]. Building on the success of BERT [37] and GPT [38] in natural language processing, researchers have developed specialized models that treat code as a sequence of tokens and leverage pretraining strategies such as masked language modeling, span corruption, and causal language modeling [6, 4, 39, 40, 31, 33, 41, 42, 43, 44, 45, 46, 47, 48].

Table 1: Comparison of features between CodeTF and HuggingFace Transformers (HF-T). Note that we compare these libraries by features related to the code domain, highlighting functionalities that HF-T may not specifically support.

Feature	CodeTF (Ours)	HF-T
Unified Model and Dataset Interface	✓	✓
Unified Parameter-Efficient Fine-Tuning for Code Intelligence Tasks	✓	✓
Unified Code Utility Interface for Multiple Programming Languages	✓	✗
Unified Metric Interface to Evaluate Code Intelligence Benchmarks	✓	✗
Unified Data Loader Interface to Process Code Intelligence Benchmarks	✓	✗
Modular Library Design	✓	✓
Pretrained Model Checkpoints	✓	✓
Task-specific Fine-tuned Model Checkpoints	✓	✓

Code LLMs can be categorized into three primary architectures: encoder-only models [6, 7, 49], decoder-only models [19, 13, 17, 5], and encoder-decoder models [50, 4, 51, 52]. Encoder-only models excel at understanding tasks such as code retrieval [15] and clone detection, while decoder-only models are well-suited for generation tasks like program synthesis [13, 14]. Encoder-decoder models [4, 50] offer versatility for both understanding and generation but may not always outperform specialized architectures on individual tasks. CodeTF supports all three architecture families, enabling users to select the most appropriate model for their specific use case.

Commercial Code Assistants. The commercial impact of Code LLMs has been transformative. GitHub Copilot, powered by OpenAI's Codex model, was launched in 2021 and has been adopted by millions of developers, demonstrating significant productivity improvements in real-world software development. Amazon CodeWhisperer and Google's code-assist tools offer similar capabilities for their respective cloud ecosystems. Tabnine provides IDE-integrated code completion using both cloud and local models, while Replit's Ghostwriter focuses on the educational and hobby developer market.

While these commercial tools have proven the value of Code LLMs in practice, they present limitations for research and customization: (1) they are closed-source, preventing examination or modification of their underlying models; (2) they do not support fine-tuning on custom datasets or domains; (3) they lack transparency in evaluation, making it difficult to compare with published research results. CodeTF addresses these limitations by providing an open-source framework with full access to model weights, training pipelines, and evaluation code.

Open-Source Libraries for Code Intelligence. Several open-source libraries have been developed to support Code LLM research and development. NaturalCC [53] is a platform designed to facilitate NLP-based code analysis research, offering utilities for training and reproduction. However, its usability is limited by complex dependencies and configuration requirements.

HuggingFace Transformers [54] is the most widely-used library for working with pretrained language models, offering user-friendly interfaces across multiple domains (vision, language, code, time series). While HuggingFace provides excellent general infrastructure, it lacks code-specific features such as AST parsing, code-aware evaluation metrics, and specialized preprocessing utilities.

Other repositories such as CodeT5 [4], CodeGeeX [55], CodeBERT [6], and CodeXGLUE [19] provide model implementations and benchmark code, but these are typically model-specific rather than unified libraries. The BigCode project has made valuable contributions through open-source models (StarCoder, SantaCoder) and datasets (The Stack), but does not provide a unified training and evaluation framework.

Positioning of CodeTF. Table 5 summarizes the comparison between CodeTF and HuggingFace Transformers for code intelligence tasks. While HuggingFace Transformers provides comprehensive general-purpose infrastructure, CodeTF offers specialized features for the code domain: a code utility interface with built-in AST parsing for 15+ languages, code attribute extraction, and language-specific preprocessing capabilities not available in general NLP libraries; standardized implementations of code-specific evaluation metrics (pass@k, CodeBLEU, Edit Similarity) with benchmark-specific evaluation scripts; and pre-built data loaders for HumanEval, MBPP, APPS, CodeXGLUE, and The Vault with consistent interfaces. CodeTF is designed as a complementary layer built upon HuggingFace Transformers rather than a replacement, leveraging its robust model infrastructure while adding code-specific capabilities that simplify code intelligence research and development.

6 Future Plans and Improvements

We continue to actively improve CodeTF as a one-stop open-source library for Code LLMs and code intelligence tasks. We have several plans to expand its capabilities and support more advanced use cases and improve model reproducibility. Some key features we aim to incorporate in the future include:

- Implementing 4-bit quantization as part of the pretrained and fine-tuned models, enabling even large models such as InstructCodeT5+ [12] to run efficiently on commercial laptops or workstations.
- Conducting comprehensive evaluations of well-known code intelligence tasks on established benchmarks (CodeXGLUE, MBPP, HumanEval, and APPS). Due to rapid advancements in the field, there is a lack of reproducibility in the performance of state-of-the-art models, making it challenging for the research community to adapt and foster collaboration.
- Enhancing the Code Utility module by adding support for other programming languages, such as Go, Rust, C#, and more. We also plan to include utilities for extracting additional useful features from code, such as call graphs, control flow, data flow, and others.
- Integrating a broader selection of recent state-of-the-art pretrained language models of code into CodeTF, further solidifying our library as a comprehensive resource in the field.

7 Conclusion

We have presented CodeTF, a comprehensive open-source library for code intelligence that addresses the fragmented landscape of Code LLM development and deployment. By providing unified interfaces for model loading, training, evaluation, and code-specific utilities, CodeTF significantly reduces the engineering effort required to work with state-of-the-art Code LLMs.

The key technical contributions of CodeTF include a unified model interface supporting 9+ pretrained models across three architectures (encoder-only, decoder-only, encoder-decoder) with seamless quantization for efficient deployment; comprehensive code utilities including AST parsing for 15+ programming languages, enabling code attribute extraction and language-aware preprocessing without external dependencies; standardized evaluation infrastructure with built-in support for popular benchmarks (HumanEval, MBPP, APPS, CodeXGLUE) and consistent metric implementations (pass@k, CodeBLEU, Edit Similarity); and parameter-efficient fine-tuning integration through HuggingFace PEFT, enabling adaptation of large models on consumer hardware.

CodeTF is designed to serve both the research community exploring new Code LLM architectures and practitioners deploying models in production systems. We believe that providing standardized, reproducible infrastructure will accelerate progress in code intelligence research by enabling fair comparisons across models and reducing the barrier to entry for new researchers.

We invite the community to contribute to CodeTF's continued development. Contributions may include adding support for new models and benchmarks, implementing additional programming language parsers, improving documentation, and sharing evaluation results. The library is available at `https://github.com/salesforce/CodeTF` under an open-source license.

As Code LLMs continue to advance rapidly, we are committed to maintaining CodeTF as a comprehensive and up-to-date resource. Future development will focus on integrating emerging models, expanding language support, and adding new evaluation capabilities as the field evolves.

8 Broader Impact and Responsible Use

While models within CodeTF show immense potential in various code-related tasks, they do not provide absolute guarantees regarding their code intelligence capabilities. The datasets and pretrained models used in CodeTF may carry biases that could result in misinterpretations, incorrect results, or undesired behaviors. These biases can take multiple forms:

1. **Language Bias**: The model might prefer certain programming languages over others based on the frequency of the languages in the training data. For instance, if the model is trained mostly on Python code, it might struggle to generate accurate and idiomatic Java or JavaScript code.

2. **Application-specific Bias**: This occurs when a model trained for a particular application or domain is used in a different application. For example, a model trained on web development code may perform poorly when tasked with generating embedded system code.
3. **Library and Framework Bias**: This refers to the inherent inclination of a model towards using specific libraries or frameworks due to the frequency of their presence in the training dataset. For example, if the model was predominantly trained on data using Python's Pandas for data manipulation, it may be more inclined to use Pandas even in situations where other libraries like NumPy or native Python constructs could be more efficient or appropriate.
4. **Language Version Bias**: Software languages evolve, with new versions (e.g., Python 2 to Python 3) introducing changes, depreciations, and novel features. If the training dataset is not updated regularly to reflect these changes, the model could generate code using outdated or deprecated conventions of a language.
5. **Coding Style Bias**: Coding style can vary significantly between individual coders, teams, or communities. If the model is trained predominantly on a dataset reflecting a specific style, it may generate code that is in accordance with that style, which may not be the optimal or preferred way for the specific use-case at hand.
6. **Solution Bias**: There can often be more than one valid solution to a coding problem. The model might be biased towards the solutions it was exposed to during training and might fail to generate other potentially more efficient or elegant solutions.

In addition to these potential biases, there are several other crucial considerations:

1. **Sustainability**: Energy efficiency is a significant concern in AI, especially with large-scale models. Optimized models generating more efficient code could reduce the computational resources required to execute such code, thereby reducing energy consumption. Ongoing research into more energy-efficient AI training methods can also decrease the energy footprint of AI itself.
2. **Inclusive language**: Coding language needs to be inclusive as the field becomes increasingly diverse. Non-inclusive terms may discourage and offend many developers. Future work should focus on creating tools to identify non-inclusive language in code and recommend suitable alternatives.
3. **Job loss and automation**: While AI carries the potential to automate certain tasks, it is essential to view it as a tool that augments rather than replaces human efforts. Developer tools are usually designed to handle repetitive tasks, freeing developers to focus on complex issues. However, it's crucial to ensure developers do not become overly reliant on these tools and can still code effectively on their own.
4. **Human control and autonomy**: Maintaining human control and oversight is crucial, especially in critical areas like code generation. Techniques like explainability and interpretability in AI, along with rigorous testing, ensure AI systems remain under human control and behave as expected. The goal should be to create AI systems that enhance human capabilities and work collaboratively with humans, rather than replacing them.

Users of CodeTF must scrutinize the pretrained models and the general system before their adoption in practical applications. We are committed to refining the library by identifying and addressing potential biases and inappropriate behaviors continually.

We encourage researchers, software engineers, and AI practitioners to use the library responsibly for applications that enhance software quality and developer productivity. However, CodeTF should not be used to develop code intelligence models that could lead to unethical capabilities, such as unauthorized code manipulation, privacy breaches, or the propagation of insecure coding practices.

As AI becomes more integrated into software development, it is essential to address these ethical and practical considerations. CodeTF is committed to supporting responsible AI practices and mitigating potential biases and inappropriate behaviors moving forward.

References

[1] Simone Livieri, Yoshiki Higo, Makoto Matushita, and Katsuro Inoue. Very-large scale code clone analysis and visualization of open source programs using distributed ccfinder: D-ccfinder. In *29th International Conference on Software Engineering (ICSE'07)*, pages 106–115. IEEE, 2007.

[2] Carol V Alexandru and Harald C Gall. Rapid multi-purpose, multi-commit code analysis. In *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*, volume 2, pages 635–638. IEEE, 2015.

[3] Boyuan Chen and Zhen Ming Jiang. Characterizing and detecting anti-patterns in the logging code. In *2017 IEEE/ACM 39th International Conference on Software Engineering (ICSE)*, pages 71–81. IEEE, 2017.

[4] Yue Wang, Weishi Wang, Shafiq R. Joty, and Steven C. H. Hoi. Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. In Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih, editors, *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, EMNLP 2021, Virtual Event / Punta Cana, Dominican Republic, 7-11 November, 2021*, pages 8696–8708. Association for Computational Linguistics, 2021.

[5] Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. Codegen: An open large language model for code with multi-turn program synthesis. In *The Eleventh International Conference on Learning Representations*, 2023.

[6] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. Codebert: A pre-trained model for programming and natural languages. In Trevor Cohn, Yulan He, and Yang Liu, editors, *Findings of the Association for Computational Linguistics: EMNLP 2020, Online Event, 16-20 November 2020*, volume EMNLP 2020 of *Findings of ACL*, pages 1536–1547. Association for Computational Linguistics, 2020.

[7] Daya Guo, Shuo Ren, Shuai Lu, Zhangyin Feng, Duyu Tang, Shujie Liu, Long Zhou, Nan Duan, Alexey Svyatkovskiy, Shengyu Fu, Michele Tufano, Shao Kun Deng, Colin B. Clement, Dawn Drain, Neel Sundaresan, Jian Yin, Daxin Jiang, and Ming Zhou. Graphcodebert: Pre-training code representations with data flow. In *ICLR*. OpenReview.net, 2021.

[8] Daya Guo, Shuai Lu, Nan Duan, Yanlin Wang, Ming Zhou, and Jian Yin. Unixcoder: Unified cross-modal pre-training for code representation. In *ACL (1)*, pages 7212–7225. Association for Computational Linguistics, 2022.

[9] Hung Le, Yue Wang, Akhilesh Deepak Gotmare, Silvio Savarese, and Steven Hoi. CodeRL: Mastering code generation through pretrained models and deep reinforcement learning. In Alice H. Oh, Alekh Agarwal, Danielle Belgrave, and Kyunghyun Cho, editors, *Advances in Neural Information Processing Systems*, 2022.

[10] OpenAI. Gpt-4 technical report. *ArXiv*, abs/2303.08774, 2023.

[11] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, et al. Starcoder: may the source be with you! *arXiv preprint arXiv:2305.06161*, 2023.

[12] Yue Wang, Hung Le, Akhilesh Deepak Gotmare, Nghi DQ Bui, Junnan Li, and Steven CH Hoi. Codet5+: Open code large language models for code understanding and generation. *arXiv preprint arXiv:2305.07922*, 2023.

[13] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[14] Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, and Jacob Steinhardt. Measuring coding challenge competence with apps. *NeurIPS*, 2021.

[15] Hamel Husain, Ho-Hsiang Wu, Tiferet Gazit, Miltiadis Allamanis, and Marc Brockschmidt. Codesearchnet challenge: Evaluating the state of semantic code search. *CoRR*, abs/1909.09436, 2019.

[16] Frank F Xu, Uri Alon, Graham Neubig, and Vincent Josua Hellendoorn. A systematic evaluation of large language models of code. In *Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming*, pages 1–10, 2022.

[17] Daniel Fried, Armen Aghajanyan, Jessy Lin, Sida Wang, Eric Wallace, Freda Shi, Ruiqi Zhong, Wen-tau Yih, Luke Zettlemoyer, and Mike Lewis. Incoder: A generative model for code infilling and synthesis. *arXiv preprint arXiv:2204.05999*, 2022.

[18] Loubna Ben Allal, Raymond Li, Denis Kocetkov, Chenghao Mou, Christopher Akiki, Carlos Munoz Ferrandis, Niklas Muennighoff, Mayank Mishra, Alex Gu, Manan Dey, et al. Santacoder: don't reach for the stars! *arXiv preprint arXiv:2301.03988*, 2023.

[19] Shuai Lu, Daya Guo, Shuo Ren, Junjie Huang, Alexey Svyatkovskiy, Ambrosio Blanco, Colin B. Clement, Dawn Drain, Daxin Jiang, Duyu Tang, Ge Li, Lidong Zhou, Linjun Shou, Long Zhou, Michele Tufano, Ming Gong, Ming Zhou, Nan Duan, Neel Sundaresan, Shao Kun Deng, Shengyu Fu, and Shujie Liu. Codexglue: A machine learning benchmark dataset for code understanding and generation. In *NeurIPS Datasets and Benchmarks*, 2021.

[20] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.

[21] Dung Nguyen, Le Nam, Anh Dau, Anh Nguyen, Khanh Nghiem, Jin Guo, and Nghi Bui. The vault: A comprehensive multilingual dataset for advancing code understanding and generation. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 4763–4788, 2023.

[22] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. Llm. int8 (): 8-bit matrix multiplication for transformers at scale. *arXiv preprint arXiv:2208.07339*, 2022.

[23] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. Gptq: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323*, 2022.

[24] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.

[25] Xiang Lisa Li and Percy Liang. Prefix-tuning: Optimizing continuous prompts for generation. *arXiv preprint arXiv:2101.00190*, 2021.

[26] Xiao Liu, Yanan Zheng, Zhengxiao Du, Ming Ding, Yujie Qian, Zhilin Yang, and Jie Tang. Gpt understands, too. *arXiv preprint arXiv:2103.10385*, 2021.

[27] Qingru Zhang, Minshuo Chen, Alexander Bukharin, Pengcheng He, Yu Cheng, Weizhu Chen, and Tuo Zhao. Adaptive budget allocation for parameter-efficient fine-tuning. *arXiv preprint arXiv:2303.10512*, 2023.

[28] Yujia Li, David H. Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, Thomas Hubert, Peter Choy, Cyprien de Masson d'Autume, Igor Babuschkin, Xinyun Chen, Po-Sen Huang, Johannes Welbl, Sven Gowal, Alexey Cherepanov, James Molloy, Daniel J. Mankowitz, Esme Sutherland Robson, Pushmeet Kohli, Nando de Freitas, Koray Kavukcuoglu, and Oriol Vinyals. Competition-level code generation with alphacode. *CoRR*, abs/2203.07814, 2022.

[29] Shuo Ren, Daya Guo, Shuai Lu, Long Zhou, Shujie Liu, Duyu Tang, Neel Sundaresan, Ming Zhou, Ambrosio Blanco, and Shuai Ma. Codebleu: a method for automatic evaluation of code synthesis. *CoRR*, abs/2009.10297, 2020.

[30] Fengji Zhang, Bei Chen, Yue Zhang, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Weizhu Chen. Repocoder: Repository-level code completion through iterative retrieval and generation. *arXiv preprint arXiv:2303.12570*, 2023.

[31] Ahmed Elnaggar, Wei Ding, Llion Jones, Tom Gibbs, Tamas Feher, Christoph Angerer, Silvia Severini, Florian Matthes, and Burkhard Rost. Codetrans: Towards cracking the language of silicon's code through self-supervised deep learning and high performance computing. *arXiv preprint arXiv:2104.02443*, 2021.

[32] Hung Q To, Nghi DQ Bui, Jin Guo, and Tien N Nguyen. Better language models of code through self-improvement. *arXiv preprint arXiv:2304.01228*, 2023.

[33] Dinglan Peng, Shuxin Zheng, Yatao Li, Guolin Ke, Di He, and Tie-Yan Liu. How could neural networks understand programs? In *International Conference on Machine Learning*, pages 8476–8486. PMLR, 2021.

[34] Chunqiu Steven Xia, Yuxiang Wei, and Lingming Zhang. Practical program repair in the era of large pre-trained language models. *arXiv preprint arXiv:2210.14179*, 2022.

[35] Baptiste Roziere, Marie-Anne Lachaux, Lowik Chanussot, and Guillaume Lample. Unsupervised translation of programming languages. *Advances in Neural Information Processing Systems*, 33:20601–20611, 2020.

[36] Nghi Bui. Towards zero knowledge learning for cross language api mappings. In *2019 IEEE/ACM 41st International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*, pages 123–125. IEEE, 2019.

[37] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2019, Minneapolis, MN, USA, June 2-7, 2019, Volume 1 (Long and Short Papers)*, pages 4171–4186, 2019.

[38] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.

[39] Daya Guo, Shuo Ren, Shuai Lu, Zhangyin Feng, Duyu Tang, Shujie Liu, Long Zhou, Nan Duan, Alexey Svyatkovskiy, Shengyu Fu, et al. Graphcodebert: Pre-training code representations with data flow. *arXiv preprint arXiv:2009.08366*, 2020.

[40] Wasi Uddin Ahmad, Saikat Chakraborty, Baishakhi Ray, and Kai-Wei Chang. Unified Pre-training for Program Understanding and Generation. In Kristina Toutanova, Anna Rumshisky, Luke Zettlemoyer, Dilek Hakkani-Tür, Iz Beltagy, Steven Bethard, Ryan Cotterell, Tanmoy Chakraborty, and Yichao Zhou, editors, *Proceedings*

of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2021, Online, June 6-11, 2021, pages 2655–2668. Association for Computational Linguistics, 2021.

[41] Aditya Kanade, Petros Maniatis, Gogul Balakrishnan, and Kensen Shi. Learning and evaluating contextual embedding of source code. In *International Conference on Machine Learning*, pages 5110–5121. PMLR, 2020.

[42] Saikat Chakraborty, Toufique Ahmed, Yangruibo Ding, Premkumar T Devanbu, and Baishakhi Ray. Natgen: generative pre-training by "naturalizing" source code. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pages 18–30, 2022.

[43] Toufique Ahmed and Premkumar Devanbu. Multilingual training for software engineering. In *Proceedings of the 44th International Conference on Software Engineering*, pages 1443–1455, 2022.

[44] Changan Niu, Chuanyi Li, Vincent Ng, Jidong Ge, Liguo Huang, and Bin Luo. Spt-code: sequence-to-sequence pre-training for learning source code representations. In *Proceedings of the 44th International Conference on Software Engineering*, pages 2006–2018, 2022.

[45] Nghi DQ Bui, Yue Wang, and Steven Hoi. Detect-localize-repair: A unified framework for learning to debug with codet5. *arXiv preprint arXiv:2211.14875*, 2022.

[46] Nghi DQ Bui, Yijun Yu, and Lingxiao Jiang. Self-supervised contrastive learning for code retrieval and summarization via semantic-preserving transformations. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 511–521, 2021.

[47] Nghi DQ Bui, Yijun Yu, and Lingxiao Jiang. Infercode: Self-supervised learning of code representations by predicting subtrees. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*, pages 1186–1197. IEEE, 2021.

[48] Minh Huynh Nguyen, Nghi DQ Bui, Truong Son Hy, Long Tran-Thanh, and Tien N Nguyen. Hierarchynet: Learning to summarize source code with heterogeneous representations. *arXiv e-prints*, pages arXiv–2205, 2022.

[49] Xin Wang, Yasheng Wang, Yao Wan, Jiawei Wang, Pingyi Zhou, Li Li, Hao Wu, and Jin Liu. CODE-MVP: Learning to represent source code from multiple views with contrastive pre-training. In *Findings of the Association for Computational Linguistics: NAACL 2022*, pages 1066–1077, Seattle, United States, July 2022. Association for Computational Linguistics.

[50] Wasi Uddin Ahmad, Saikat Chakraborty, Baishakhi Ray, and Kai-Wei Chang. Unified pre-training for program understanding and generation. In *NAACL-HLT*, pages 2655–2668. Association for Computational Linguistics, 2021.

[51] Changan Niu, Chuanyi Li, Vincent Ng, Jidong Ge, Liguo Huang, and Bin Luo. Spt-code: Sequence-to-sequence pre-training for learning source code representations. In *ICSE*, pages 1–13. ACM, 2022.

[52] Saikat Chakraborty, Toufique Ahmed, Yangruibo Ding, Prem Devanbu, and Baishakhi Ray. Natgen: generative pre-training by "naturalizing" source code. *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2022.

[53] Yao Wan, Yang He, Zhangqian Bi, Jianguo Zhang, Yulei Sui, Hongyu Zhang, Kazuma Hashimoto, Hai Jin, Guandong Xu, Caiming Xiong, et al. Naturalcc: an open-source toolkit for code intelligence. In *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings*, pages 149–153, 2022.

[54] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander Rush. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45, Online, October 2020. Association for Computational Linguistics.

[55] Qinkai Zheng, Xiao Xia, Xu Zou, Yuxiao Dong, Shan Wang, Yufei Xue, Zihan Wang, Lei Shen, Andi Wang, Yang Li, Teng Su, Zhilin Yang, and Jie Tang. Codegeex: A pre-trained model for code generation with multilingual evaluations on humaneval-x, 2023.